

Technische Hochschule Ingolstadt
Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

IMU Fusion Recap
Sensing with Cameras

Oct 02, 2025

Welcome to the lecture!

Prof. Dr.-Ing. Björn Giesler

Technische Hochschule Ingolstadt / Fraunhofer IVI

Autonomous, Cooperating Systems

Email: bjoern.giesler@thi.de

- **Please register for the lecture on Moodle!**
  - Link: https://moodle.thi.de/course/view.php?id=10580
  - Key: **SensNT+DF#WS25**
- **Materials provided on Moodle starting on Monday, Oct 06:**
  - Slides for the lecture
  - Formulary
  - Python code
  - Exercises / Solutions provided after discussion in the lecture
  - Exercises are optional, one sheet per week, but are highly recommended.
    They can be handed in for correction.
- **Exam: Written exam, 90 minutes**
- **Consultation hours: Please arrange via e-mail to bjoern.giesler@thi.de.**
- **Please also don‘t hesitate to send questions, lecture feedback, change requests, etc.!**

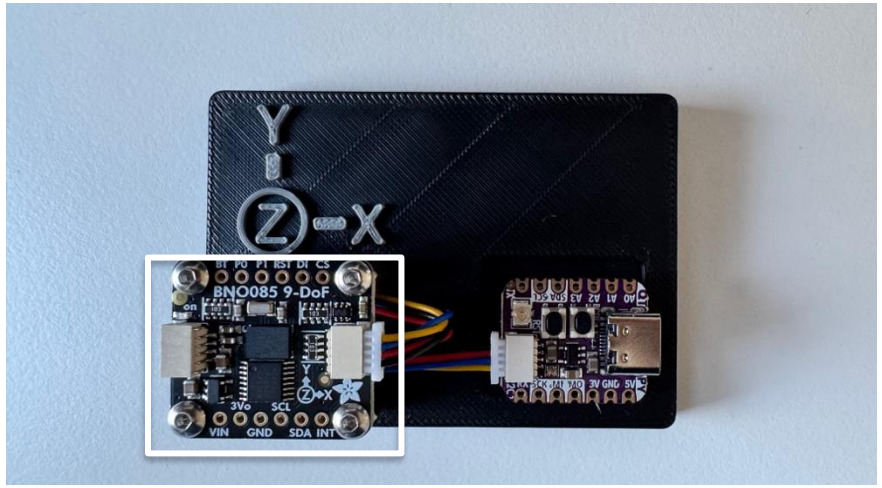
- **IMU Fusion – Complementary Filter**
- **Sensing with Cameras**
  - Projective Properties
  - Linear Calibration
  - Object Detection using Viola-Jones

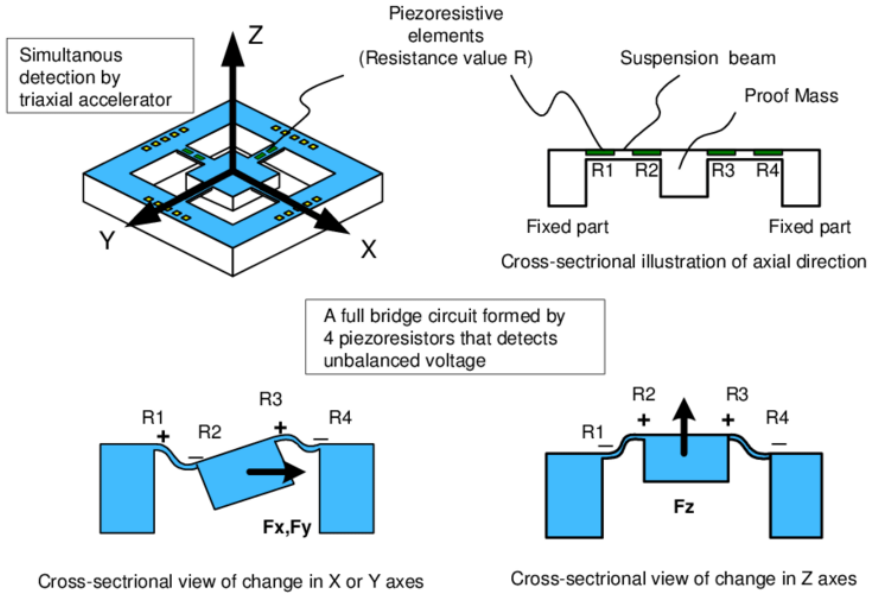
Continuation – Pitch and Roll from Gyro and Accelerometer

(with Complementary Filters, for now)

Sensor Networks and Sensor Fusion

Simple Example: Inertial Measurement Unit





Derive Pitch and Roll from Accelerometer / Gyro Fusion!

Accelerometer (at rest, ignoring motion, bias and noise):

$$\begin{pmatrix} a_x \\ a_y \\ a_z \end{pmatrix} = \begin{pmatrix} g\sin\beta \\ -g\sin\alpha\cos\beta \\ g\cos\alpha\cos\beta \end{pmatrix} \Rightarrow \quad \hat{\alpha}_{\text{acc}} = \tan^{-1}\frac{a_y}{a_z}, \quad \hat{\beta}_{acc} = \sin^{-1}\frac{a_x}{g}$$

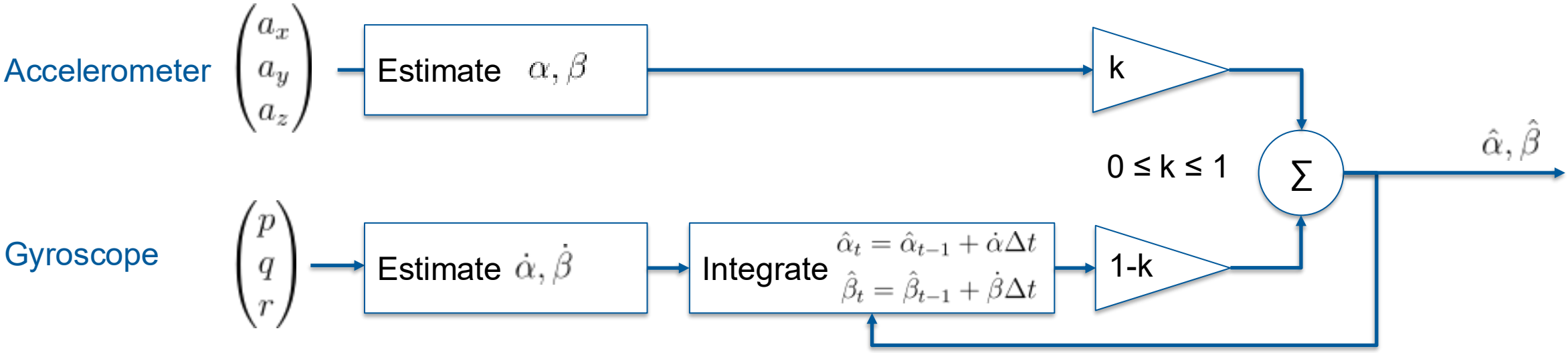
Gyro (ignoring bias and noise):

$$\begin{pmatrix} \dot{\alpha} \\ \dot{\beta} \end{pmatrix} = \begin{pmatrix} 1 & \sin\alpha\tan\beta & \cos\alpha\tan\beta \\ 0 & \cos\alpha & -\sin\alpha \end{pmatrix} \begin{pmatrix} p \\ q \\ r \end{pmatrix}$$

- **What have we learned?**
- **Both accelerometer and gyroscope can give estimates of pitch and roll at certain points.**
  - Accelerometers give good estimates when at rest or over very long periods of time, when accelerations average out and only gravity is left.
  - Gyroscopes give good estimates over very short periods of time, when noise and bias have no time to accumulate.
- **We cannot solve the problem with either of the sensors alone. We need to fuse the data.**

- **Fusing accelerometers and gyroscopes is a well studied problem since the first use of gyroscopes in WW2 rockets.**
- **Many methods have been developed, in order of ascending complexity:**
  - Complementary filters
  - Madgwick or Mahony filters, combining complementary filters with gradient-descent optimization
  - Kalman filters and Extended Kalman filters
  - Neural networks trained on accel/gyro data for various motion profiles.
- **We will look at all of these methods during the course of the lecture (although not always with an IMU in mind).**
- **Today we will look at the simplest variant, the Complementary Filter.**

- **Complementary filters are one of the simplest and most computationally effective fusion methods.**
- **The idea is to use the output of one sensor to correct errors in the output of the other, and vice versa.**
- **Basic principle for our sensor: Weighted sum of individual estimates**



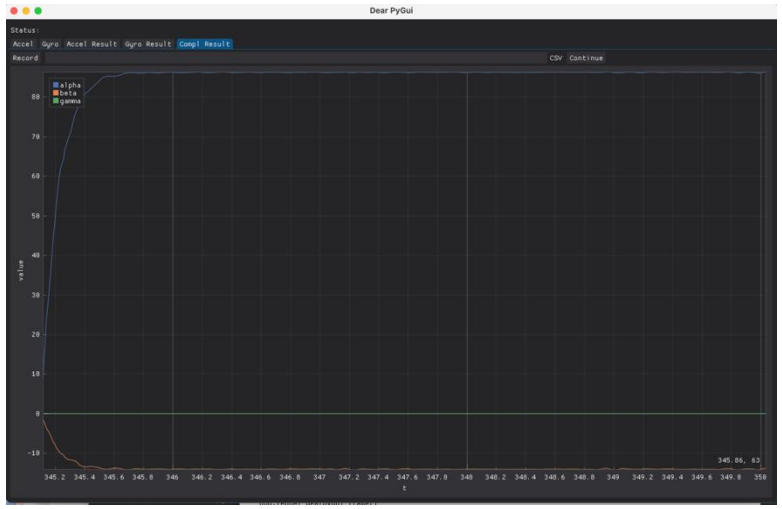
- **Complementary Filter equation:**

$$\hat{\alpha}_n = k\hat{\alpha}_{\text{acc},n} + (1-k)(\hat{\alpha}_{n-1} + \dot{\alpha}_{\text{gyro},n}\Delta t)$$
$$\hat{\beta}n = k\hat{\beta}_{\text{acc},n} + (1-k)(\hat{\beta}_{n-1} + \dot{\beta}_{\text{gyro},n}\Delta t)$$

- **k is a constant which roughly says how much we trust our accelerometer. It is typically close to 0 but we can and should experiment with it.**
- **Let‘s see how this works!**

Sensor Networks and Sensor Fusion

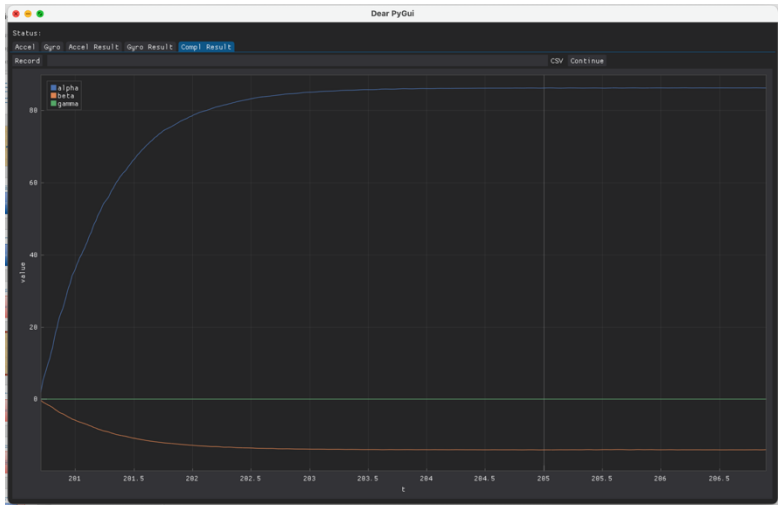
Different values for k



0.1



0.05



0.02

- **An alternative way of thinking about the Complementary Filter:**
- **We can think of the development of the state of a system that follows a linear, time-invariant model like this:**

$$\mathbf{x}_n = \mathbf{A}\mathbf{x}_{n-1} + \mathbf{B}\mathbf{u}_{n-1}$$

where

  - x is the state vector,
  - A is the system model, i.e. “the way things change in the system over time if there is no input“,
  - u is a vector of inputs to the system,
  - and B is the way how these inputs are influencing our system state.
- **If there were no noise and we could just integrate the gyro, our model can be written as**

$$\underbrace{\begin{pmatrix}\hat{\alpha}_n\\ \hat{\beta}_n\end{pmatrix}}_{\mathbf{x}_n} = \underbrace{\begin{pmatrix}1 & 0\\ 0 & 1\end{pmatrix}}_{\mathbf{A}} \underbrace{\begin{pmatrix}\hat{\alpha}_{n-1}\\ \hat{\beta}_{n-1}\end{pmatrix}}_{\mathbf{x}_{n-1}} + \underbrace{\begin{pmatrix}\Delta t & 0\\ 0 & \Delta t\end{pmatrix}}_{\mathbf{B}} \underbrace{\begin{pmatrix}\dot{\alpha}_{n-1}\\ \dot{\beta}_{n-1}\end{pmatrix}}_{\mathbf{u}_{n-1}}$$

- **However, we know that that is not correct, so we introduce a correction term...**

$$\mathbf{x}_n = \mathbf{A}\mathbf{x}_{n-1} + \mathbf{B}\mathbf{u}_{n-1} + \mathbf{L}(\mathbf{x}_{n-1} - \mathbf{x}_{\mathrm{meas},\mathbf{n-1}})$$

...again with a model L that governs how the correction term applies to the system state.

- **We can write this now as**

$$\underbrace{\begin{pmatrix}\hat{\alpha}_n\\ \hat{\beta}_n\end{pmatrix}}_{\mathbf{x}_n} = \underbrace{\begin{pmatrix}1 & 0\\ 0 & 1\end{pmatrix}}_{\mathbf{A}} \underbrace{\begin{pmatrix}\hat{\alpha}_{n-1}\\ \hat{\beta}_{n-1}\end{pmatrix}}_{\mathbf{x}_{n-1}} + \underbrace{\begin{pmatrix}\Delta t(1-k) & 0\\ 0 & \Delta t(1-k)\end{pmatrix}}_{\mathbf{B}} \underbrace{\begin{pmatrix}\dot{\alpha}_{n-1}\\ \dot{\beta}_{n-1}\end{pmatrix}}_{\mathbf{u}_{n-1}} + \underbrace{\begin{pmatrix}k & 0\\ 0 & k\end{pmatrix}}_{\mathbf{L}} \begin{pmatrix}\hat{\alpha}_n - \hat{\alpha}_{\mathrm{acc},n-1}\\ \hat{\beta}_n - \hat{\beta}_{\mathrm{acc},n-1}\end{pmatrix}$$

- **The above form is known as a Luenberger Observer and is a classic in control theory.**
- **We will come back to this form of thinking about systems (state transition model, input model, correction term) *constantly.***

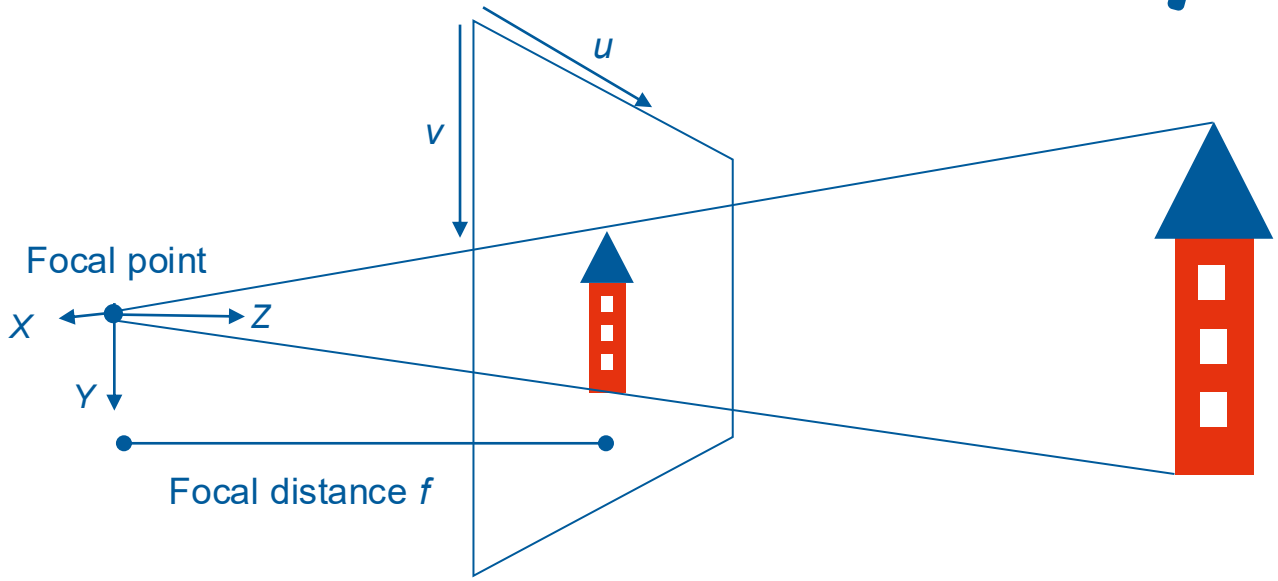
Cameras, Projection, Calibration

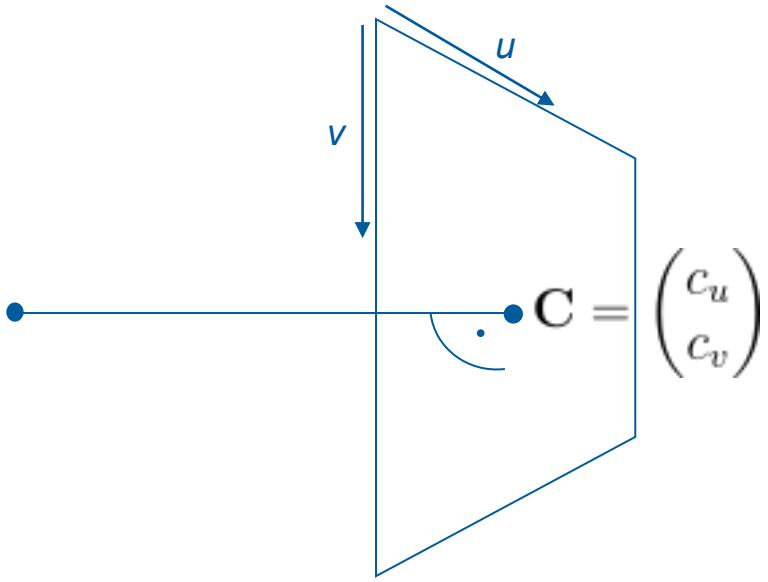
- **In this lecture we will look at cameras as our first exteroceptive sensor.**
- **Traditionally, cameras have been used in robotics and automotive applications for tasks that rely heavily on classification.**
  - Classification: The task of identifying the class of an object (vehicle, pedestrian, face, cup).
  - Detection: The task of making sure an object exists, regardless of class.
- **However, nowadays**
  - cameras are extremely cheap and ubiquitous,
  - compute constraints are much less of an issue,
  - ML and AI have overcome many shortcomings of cameras.



- **Cameras are inherently 2D sensors. The 3D environment is projected onto the imager, losing distance information.**
- **If we want to model the world with cameras,**
  - we must understand how camera projection works, and
  - we must be able to calibrate our camera for its specific calibration parameters.
- **The simplest camera model is the pinhole model, with three parameters:**
  - Focal distance *f*
  - Image center $\mathbf{C} = \begin{pmatrix} c_u \\ c_v \end{pmatrix}$

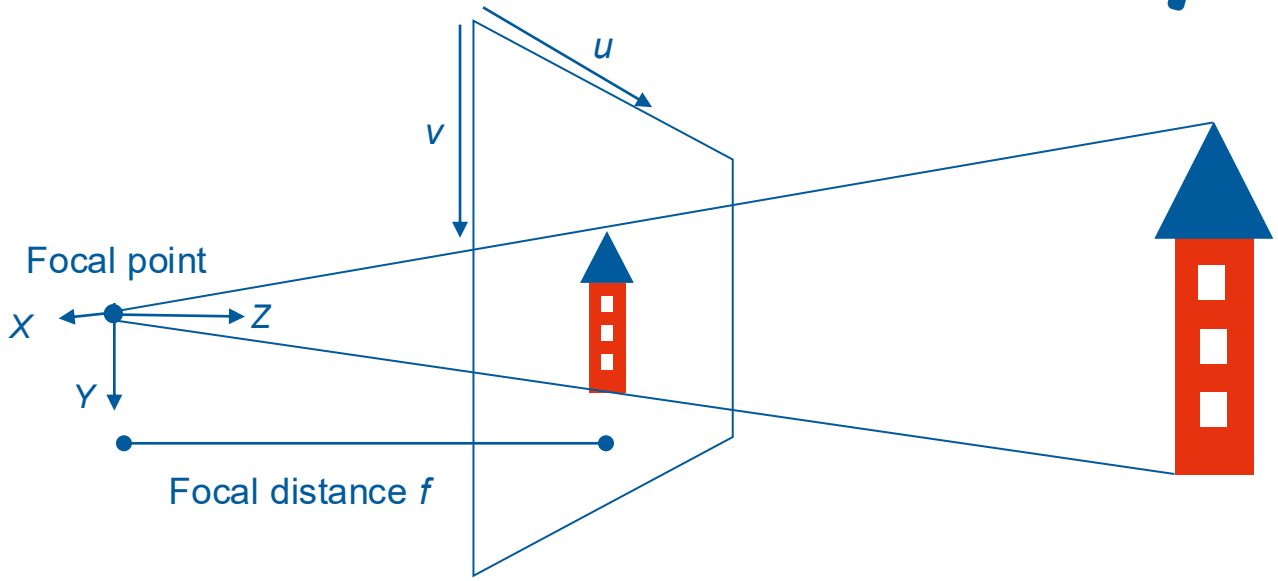
    (the point which the ray with x=0,y=0 projects to)





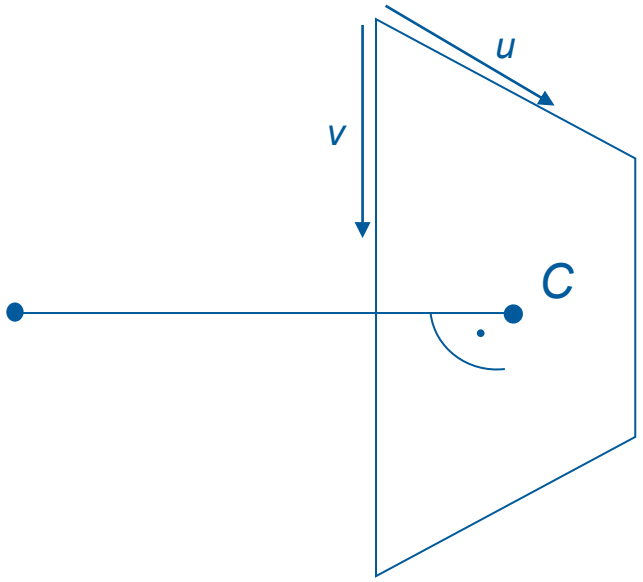
- **The basic projections in the pinhole model follow the intercept theorem („Strahlensatz"):**

$$\frac{x}{z} = \frac{u + c_u}{f} \Rightarrow u = f\frac{x}{z} - c_u$$
$$\frac{y}{z} = \frac{v + c_v}{f} \Rightarrow v = f\frac{y}{z} - c_v$$



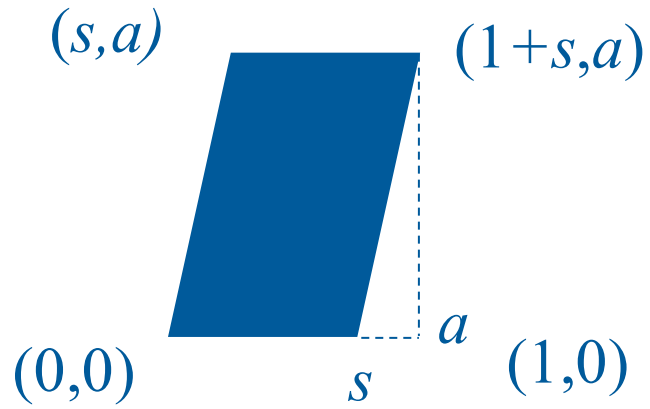
- **...or as a vector-matrix equation:**

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{z} \underbrace{\begin{pmatrix} f & 0 & -c_u \\ 0 & f & -c_v \\ 0 & 0 & 1 \end{pmatrix}}_{\mathbf{K}} \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$



- **Often camera pixels are not exactly square, especially with cheap cameras, webcams or similar..**
- **Some imagers actually have rectangular pixels.**
- **Often the imager chip is not precisely aligned with the lens.**
- **It is helpful to extend the calibration by a factor a that relates this ratio:**

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{z} \underbrace{\begin{pmatrix} f & 0 & -c_u \\ 0 & af & -c_v \\ 0 & 0 & 1 \end{pmatrix}}_{\mathbf{K}} \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$



- **The other element of camera projection is the camera‘s coordinate frame, given as**

$$\mathbf{M} = \begin{pmatrix} & \mathbf{R} & & \mathbf{t} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

  where R is the 3x3 rotation matrix and t is the 3-vector modeling the translation.

- **The complete projection equation becomes**

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{P} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = (\mathbf{KR} \quad \mathbf{Kt}) \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

- **We therefore need to calibrate 9 parameters, or 12 if we ignore the constraints contained in the rotation matrix R.**

- **To determine P, we use an easily detectable object with known 3D structure, like a checkerboard.**
- **Each snapshot of the object gives a vector**

$$\begin{pmatrix} x_1 & y_1 & z_1 & x_2 & y_2 & z_2 & \dots & x_n & y_n & z_n \end{pmatrix}^\top$$

and a corresponding vector

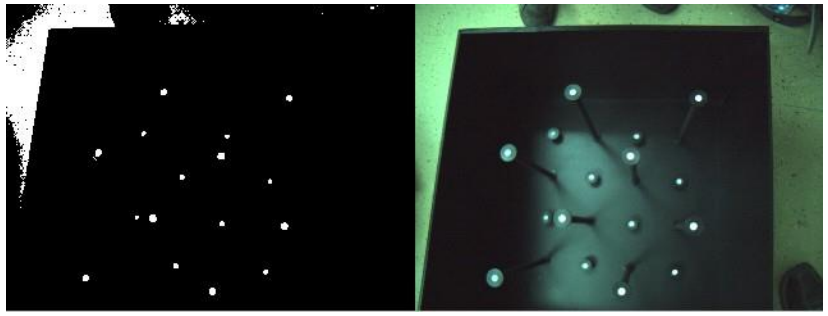
$$\begin{pmatrix} \hat{u}_1 & \hat{v}_1 & \hat{u}_2 & \hat{v}_2 & \dots & \hat{u}_n & \hat{v}_n \end{pmatrix}^\top$$

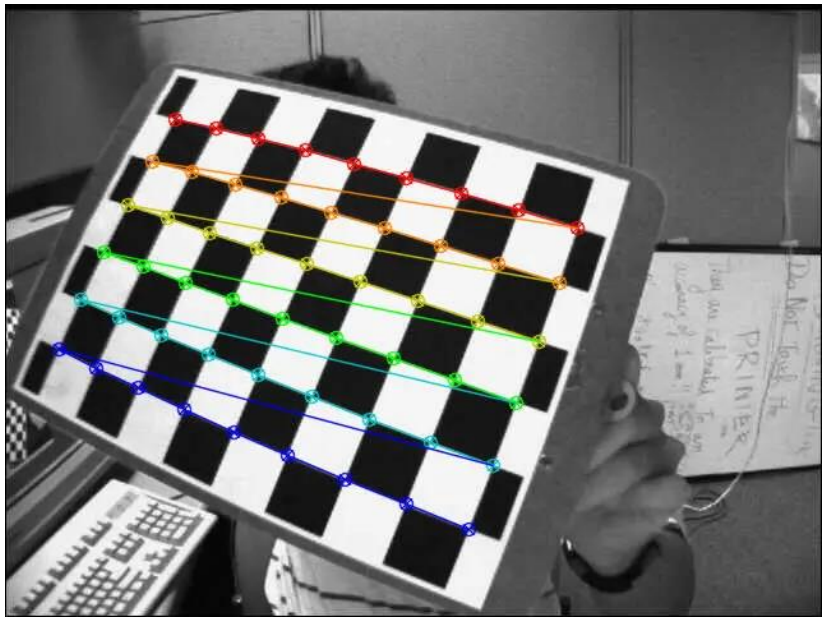
- **During projection and detection, we are adding error terms to the detection coordinates, so we need to consider**

$$\begin{pmatrix} \hat{u}_n \\ \hat{v}_n \end{pmatrix} = \begin{pmatrix} u_n \\ v_n \end{pmatrix} + \begin{pmatrix} e_{u,n} \\ e_{v,n} \end{pmatrix}$$

- **We want to find the best solution for P that minimizes the error.**







Least Squares Optimization

- **We want to solve the overdetermined system of equations $\mathbf{Ar} = \mathbf{q} + \mathbf{e}$, where**

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix}$$ **and** $$\mathbf{q} = \begin{pmatrix} q_1 \\ q_2 \\ \vdots \\ q_n \end{pmatrix}$$ **are known,**

$$\mathbf{r} = \begin{pmatrix} r_1 \\ r_2 \\ \vdots \\ r_m \end{pmatrix}$$ **is a vector of unknown parameters to be solved for, and**

$$\mathbf{e} = \begin{pmatrix} e_1 \\ e_2 \\ \vdots \\ e_m \end{pmatrix}$$ **with $\mathbf{e} = \mathbf{Ar} - \mathbf{q}$ is a vector of residual errors to be minimized.**

- **To minimize e, we introduce a function of square errors:**

$$\begin{aligned} S(\mathbf{r}) &= \sum e_i^2 = \mathbf{e}^\top \mathbf{e} = (\mathbf{Ar}-\mathbf{q})^\top(\mathbf{Ar}-\mathbf{q}) \\ &= (\mathbf{Ar})^\top \mathbf{Ar} - (\mathbf{Ar})^\top \mathbf{q} - \mathbf{q}^\top \mathbf{Ar} + \mathbf{q}^\top \mathbf{q} \\ &= (\mathbf{Ar})^\top \mathbf{Ar} - 2(\mathbf{Ar})^\top \mathbf{q} + \mathbf{q}^\top \mathbf{q} \end{aligned}$$

- **Minimization works classically by setting the derivative to zero. S is scalar valued, but r is a vector, and we derive by each component. The resulting vector of partial derivations is**

$$\begin{aligned} \frac{\partial S}{\partial \mathbf{r}} &= 2\mathbf{A}^\top \mathbf{Ar} - 2\mathbf{A}^\top \mathbf{q} = 0 \\ \mathbf{A}^\top \mathbf{Ar} &= \mathbf{A}^\top \mathbf{q} \end{aligned}$$

- **The inverse of $\mathbf{A}^\top \mathbf{A}$ generally exists if the system of equations is overdetermined, i.e. for the $m \times n$ matrix A, m>n. For this kind of problem, m is typically very much larger than n.**
- **So we can find our vector r with $\mathbf{r} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{q}$.**

- **In projective cases, we often can reconstruct only up to a scaling value.**
  We cannot tell if the camera is 100m away from a 10m high King Kong, or only 1m away from a 10cm high miniature of King Kong.
- **Linear equation systems in inhomogeneous space can have the form $\mathbf{Ar} = 0$, which has the trivial solution $\mathbf{A} = 0$ that we need to avoid.**
- **Since we can scale the system arbitrarily, we can demand to find a solution that lies on the unity sphere: $\mathbf{r}^\top \mathbf{r} = 1$. To do this, we introduce a Lagrange constraint.**
- **Lagrange constraints:**
  - The idea is to find the maximum of a function $f(x)$ under a constraint.
  - The constraint can be given as a second function, $g(x) = 0$.
  - So we try to maximize a new function, $\mathcal{L}(x, \lambda) = f(x) + \lambda g(x)$.
- **For the case of our error function S, we get $\mathcal{L}(\mathbf{r}, \lambda) = S(\mathbf{r}) + \lambda(\mathbf{r}^\top \mathbf{r} - 1)$.**

- **The new function** $\mathcal{L}(\mathbf{r}, \lambda) = S(\mathbf{r}) + \lambda(\mathbf{r}^\top \mathbf{r} - 1)$ **gets again derived by the individual elements of r.**
- **From the homogeneous case, we know that**

$$\frac{\partial S}{\partial \mathbf{r}} = 2\mathbf{A}^\top \mathbf{A}\mathbf{r} - 2\mathbf{A}^\top \mathbf{q}$$

and since q=0, this simplifies to $2\mathbf{A}^\top \mathbf{A}\mathbf{r}$ **.**

- **The full derivative of** $\mathcal{L}$ **is therefore** $\frac{\partial \mathcal{L}}{\partial \mathbf{r}} = 2\mathbf{A}^\top \mathbf{A}\mathbf{r} + 2\lambda\mathbf{r}$ **.**

- **We set to zero to find the minimum:**

$$\mathbf{A}^\top \mathbf{A}\mathbf{r} + \lambda\mathbf{r} = \mathbf{0}$$
$$\mathbf{A}^\top \mathbf{A}\mathbf{r} = -\lambda\mathbf{r}$$

- **This is an Eigenvalue problem.**

Eigenvalues and Eigenvectors

- **Eigenvector** of a matrix $\boldsymbol{A}$ corresponding to **eigenvalue** λ_i given by

$$(\boldsymbol{A} - \lambda_i \boldsymbol{I})\,\boldsymbol{v}_i = \boldsymbol{0} \quad \text{or} \quad \boldsymbol{A}\boldsymbol{v}_i = \lambda_i \boldsymbol{v}_i$$

- Multiplying from the left with $\boldsymbol{A^{-1}}$ we obtain

$$\boldsymbol{v}_i = \lambda_i \boldsymbol{A}^{-1} \boldsymbol{v}_i$$

$$\leftrightarrow \boldsymbol{A}^{-1} \boldsymbol{v}_i = \frac{1}{\lambda_i} \boldsymbol{v}_i$$

- So, the eigenvector $\boldsymbol{v}_i$ of $\boldsymbol{A}$ is also an eigenvector of $\boldsymbol{A^{-1}}$ for the eigenvalue $1/\lambda_i$.

- **The Eigenvalues of a matrix of the form $\mathbf{A}^\top\mathbf{A}$ can be derived via a numerical method called Singular Value Decomposition or SVD.**
- **We will look at properties and derivation of SVD in a later lecture. For here, let the following be sufficient:**
- **The SVD of a mxn matrix M is a factorization of the form $\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, where Σ is an $m \times n$ rectangular diagonal matrix with non-negative real numbers on the diagonal called singular values of M.**
- **For a matrix of the form $\mathbf{A}^\top\mathbf{A}$,**
  - the singular values the square roots of the Eigenvalues, and
  - the column vectors of $\mathbf{V}$ are the normalized Eigenvectors.
- **The Eigenvector corresponding to the smallest Eigenvalue is the solution we are looking for.**

- **To get our equation system of 3D points**

$$\begin{pmatrix} x_1 & y_1 & z_1 & x_2 & y_2 & z_2 & \dots & x_n & y_n & z_n \end{pmatrix}^\top$$

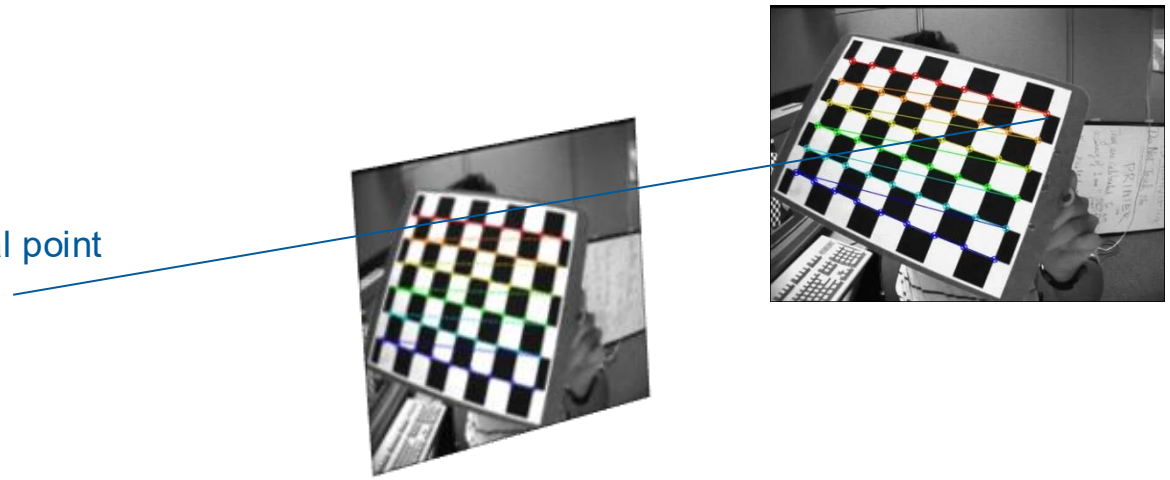
and 2D points

$$\begin{pmatrix} \hat{u}_1 & \hat{v}_1 & \hat{u}_2 & \hat{v}_2 & \dots & \hat{u}_n & \hat{v}_n \end{pmatrix}^\top$$

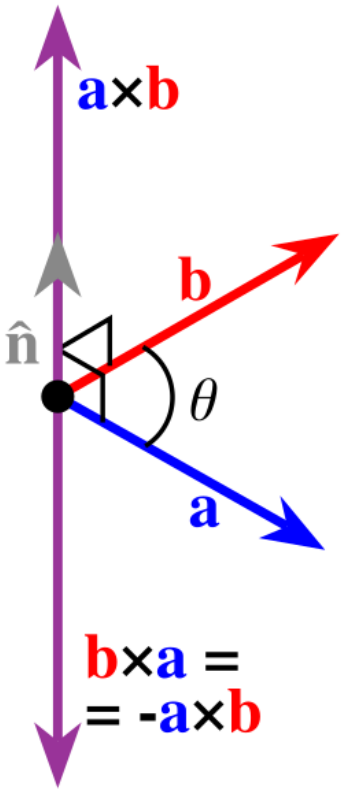
from the form $\mathbf{Px} = \mathbf{u}$

- **to the form** $\mathbf{Ar} = \mathbf{0}$

we use the fact that the points in x and the points in u each lie on the same straight line.



- **Cross Product**
- **The cross product of two vectors $\mathbf{a}$ and $\mathbf{b}$, written $\mathbf{a} \times \mathbf{b}$, is a vector that is perpendicular to the plane containing $\mathbf{a}$ and $\mathbf{b}$.**
- **It is defined by the equation $\mathbf{a} \times \mathbf{b} = |\mathbf{a}||\mathbf{b}|\mathbf{n} \sin\theta$, where n is the normal vector of the plane containing $\mathbf{a}$ and $\mathbf{b}$, and θ is the angle between the two.**
- **This is a very useful concept that e.g. can be used to determine the normal vector of a plane we know two vectors of.**
- **For collinear vectors, i.e. $\theta = 0$, the cross product is zero.**



- **We can use this constraint to reshape our equation.**

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{P} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \Leftrightarrow \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \times \mathbf{P} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \mathbf{0}$$

- **Multiplying out that equation gives us**

$$p_{11}x_n + p_{12}y_n + p_{13}z_n + p_{14} - u_n p_{31}x_n - u_n p_{32}y_n - u_n p_{33}z_n - u_n p_{34} = 0$$
$$p_{21}x_n + p_{22}y_n + p_{23}z_n + p_{24} - u_n p_{31}x_n - u_n p_{32}y_n - u_n p_{33}z_n - u_n p_{34} = 0$$
$$p_{31}x_n + p_{32}y_n + p_{33}z_n + p_{34} - p_{31} - p_{32} - p_{33} - p_{34} = 0$$

  where $p_{11} \cdots p_{34}$ **are the elements of the projection matrix P.**

- **We can write this as**

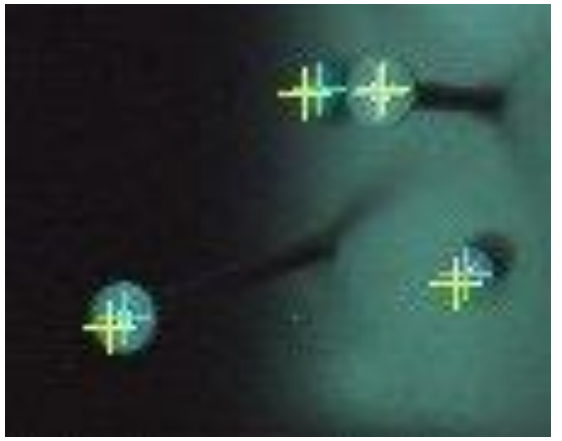
$$\mathbf{A} \begin{pmatrix} p_{11} \\ p_{12} \\ \vdots \\ p_{34} \end{pmatrix} = \mathbf{0}$$

  with the matrix A populated by $(x_n, y_n, z_n, u_n, v_n)$ **respectively.**

- **We solve for the smallest Eigenvalue as described, and rearrange for P.**

- **We measure the remaining error for the calibration by *backprojecting* the 3D points into the image and determining the backprojection error:**

$$\begin{pmatrix} u'_i \\ v_i \\ 1 \end{pmatrix} = \mathbf{P} \begin{pmatrix} x_i \\ y_i \\ z_i \\ 1 \end{pmatrix} \qquad e = \sum \sqrt{u_i^{\,2} + u'^{\,2}_i} + \sum \sqrt{v_i^{\,2} + v'^{\,2}_i}$$



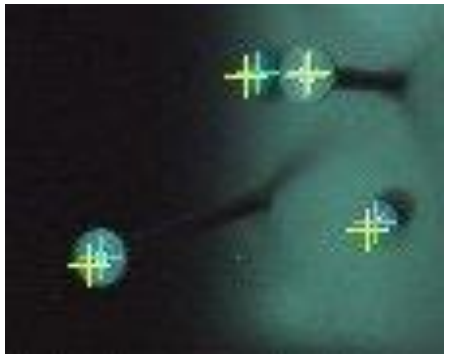
Blue: Detected points
Yellow: Backprojected points

- ***Compute Accuracy Matters!***
- **C++ floats and doubles have the highest accuracy in [-1..1]. Norming the 2D and 3D point pairs by**

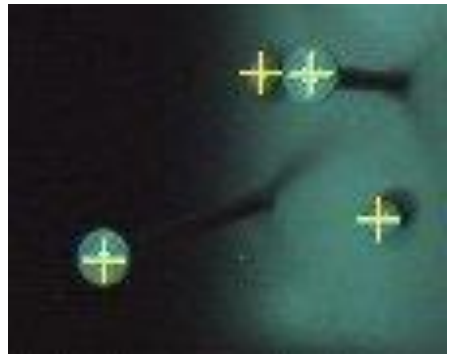
$$\nu = max_i \sqrt{u_i^2 + v_i^2}, (u'_i, v'_i) = (u_i/\nu, v_i/\nu)$$

$$\mu = max_i \sqrt{x_i^2 + y_i^2 + z_i^2}, (x'_i, y'_i, z'_i) = (x_i/\mu, y_i/\mu, z_i/\mu)$$

then performing the calibration with the normed point pairs and inflating again afterwards by multiplying with ν, μ increases accuracy tremendously.



Direct



Normalized

Backprojection results of calibration with direct and normalized point pairs

- **A couple of notes on this method for camera calibration:**
- **This gives us the projection matrix $\mathbf{P} = (\mathbf{KR} \mid \mathbf{Kt})$, not the calibration matrix and the transformation separately.**
- **K, R and t can be obtained from P using QR decomposition. We will look at this in the next lecture.**
- **So far we have looked at a purely linear pinhole camera model.**
- **Real cameras are not linear but exhibit lens distortion.**
- **This can and must be calibrated too.**
  We will look at this also in the next lecture.







Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Thank you very much!